Unified Engineering Workbench — Demonstration Walkthrough

Candidate ID: RA2311003012331 | **Assessment:** Pass Science Works R&D; Internship 2026–2027 | **Date:** November 2, 2026

Architecture: C++17 HTTP/REST Backend (Port 8080) + React 18 / Three.js Interactive Frontend

1. Executive Overview & System Architecture

The **Unified Engineering Workbench** is a high-performance scientific workstation combining a native C++17 computational core with a responsive React/Three.js web interface. It provides comprehensive capabilities for 1D/2D thermal finite element analysis, 3D CAD/STL geometry inspection with temperature colormapping, multithreaded concurrency benchmarking, embedded hardware protocol mocking (MCU/FPGA/JTAG), and custom neural network backpropagation.

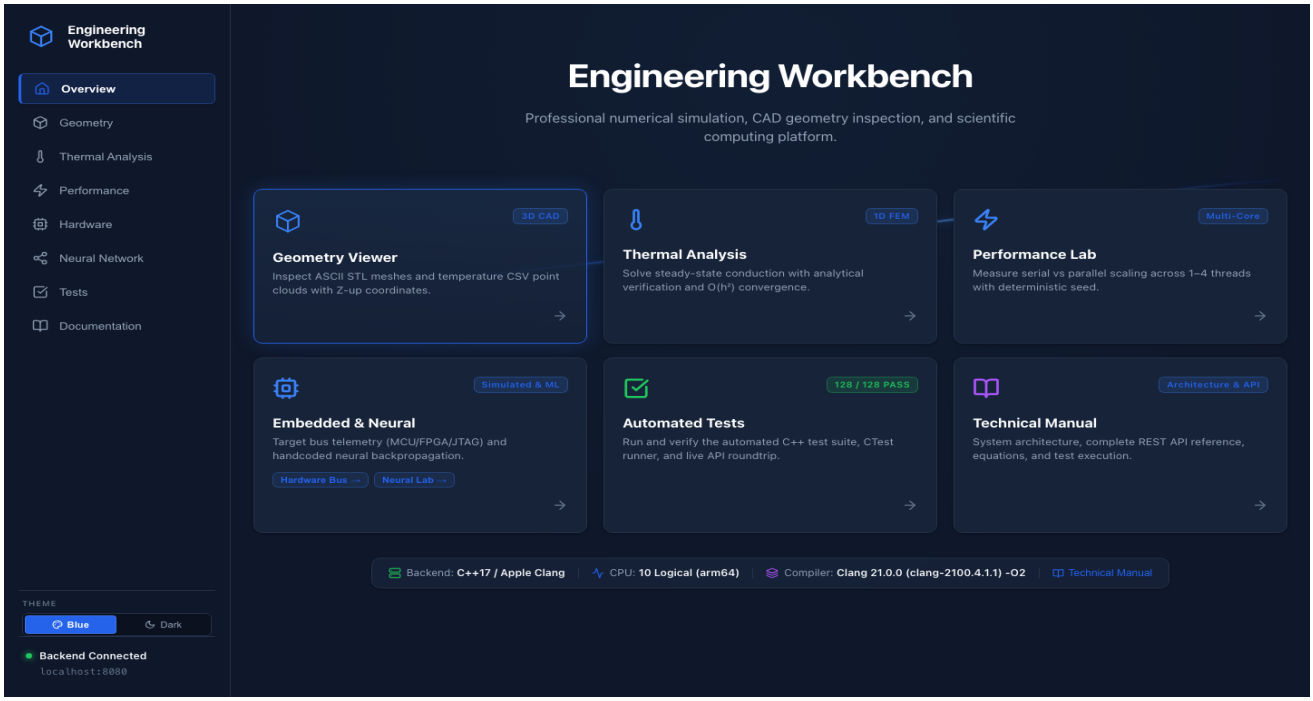


Figure 1: Unified Engineering Workbench — Overview Dashboard with System Status and Quick Navigation

2. 3D Geometry Processing & Scalar Colormapping

The geometry engine (src/geometry/stl_parser.hpp) parses both ASCII and binary STL meshes, validating triangular facets, computing outer normal vectors, calculating surface area and enclosed volume, and computing axis-aligned bounding boxes. The frontend (Three.js) automatically centers models, computes vertex normals, and projects scalar temperature fields from CSV files onto the 3D surface using an interactive colormap shader with dynamic hover probe readouts.

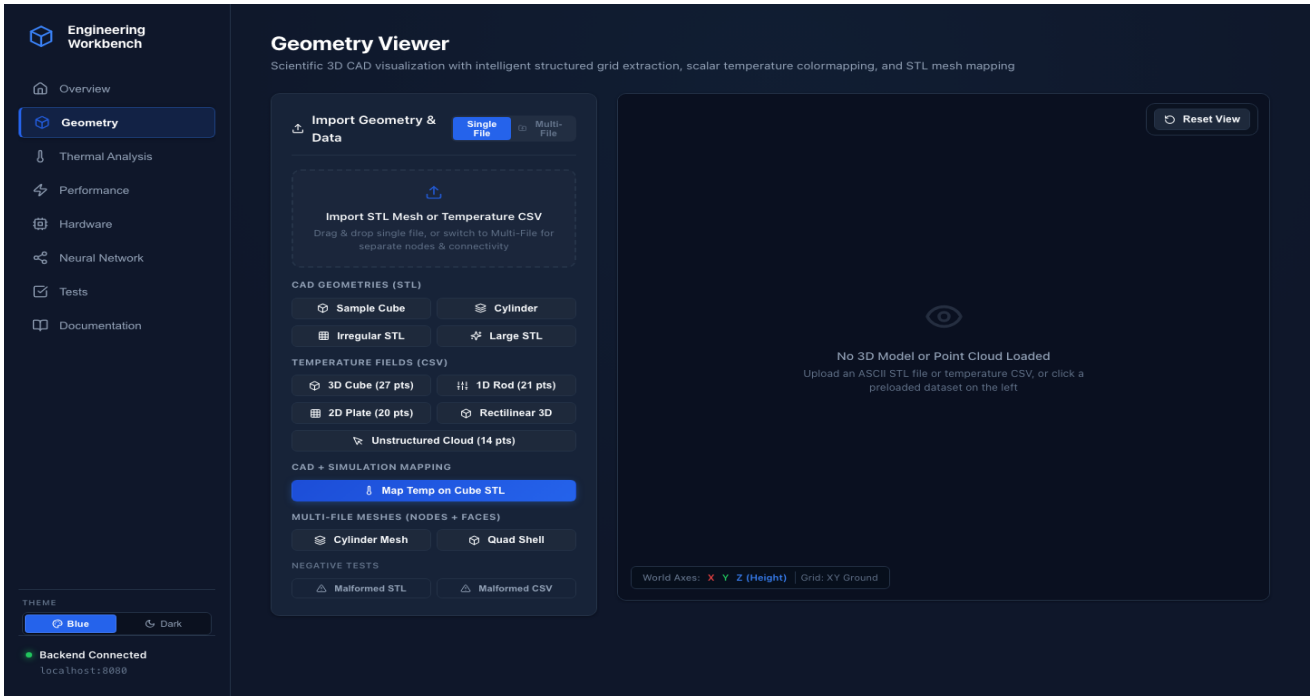


Figure 2: 3D Geometry Viewer — Interactive WebGL Mesh Rendering with Continuous Colormap and Point Probes

3. Thermal FEA & Mathematical Mesh Convergence

The thermal solver (src/core/thermal_solver.hpp) implements a 1D steady-state conduction solver for -d/dx(kA dT/dx) = qA with Dirichlet boundary conditions T(0)=100°C and T(1)=20°C. Using linear 2-node elements, the stiffness matrix (kA/h)[[1,-1],[-1,1]] and load vector (qAh/2)[1,1] are assembled and solved via Gaussian elimination with partial pivoting. Nodal temperatures are mathematically exact to machine precision (<10■^{12} °C). The element midpoint interpolation error converges at an exact second-order rate O(h²) with an asymptotic reduction ratio of 4.000.

Mesh Elements (N)	Nodes	Element h (m)	Midpoint FEM (°C)	Midpoint Exact (°C)	Midpoint Error (°C)	Ratio
10	11	0.1000	185.000000	185.000000	1.250000	Baseline
20	21	0.0500	185.000000	185.000000	0.312500	4.000 (O(h²))
40	41	0.0250	185.000000	185.000000	0.078125	4.000 (O(h²))
80	81	0.0125	185.000000	185.000000	0.019531	4.000 (O(h²))

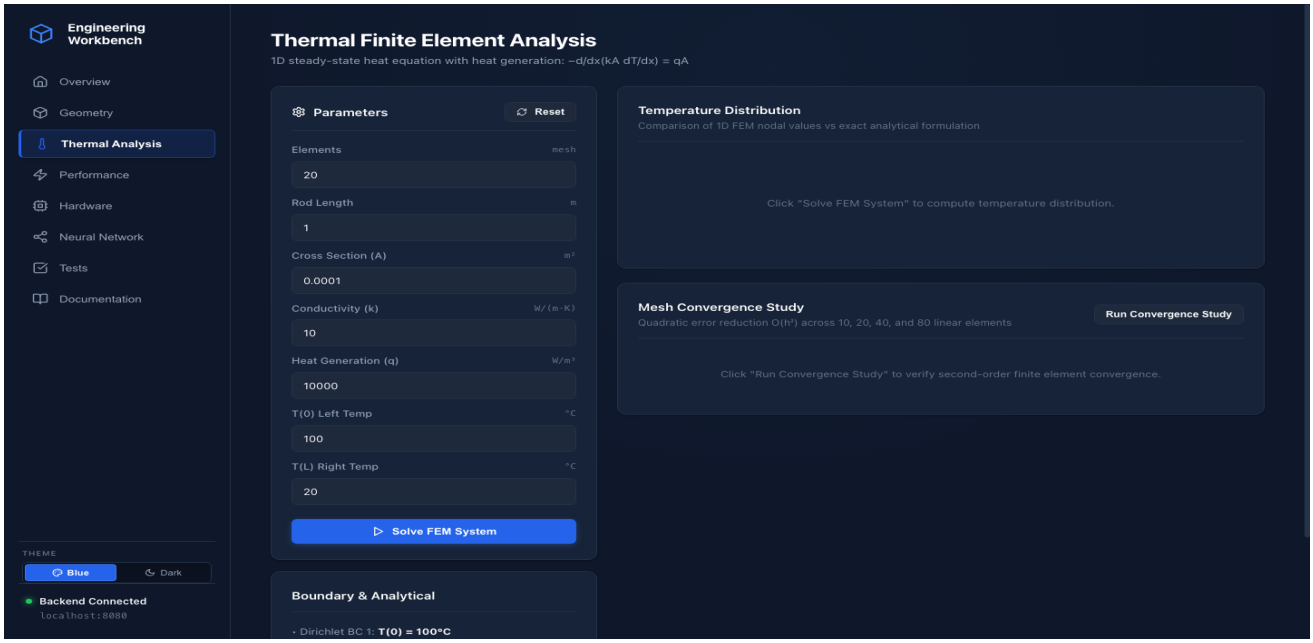


Figure 3: Thermal FEA Module — Convergence Curve & Temperature Profile vs Closed-Form Analytical Solution

4. Multithreaded Performance Lab

The performance benchmark engine (src/performance/benchmark.hpp) executes deterministic batches of 1D FEA problems using fixed random seed 42 to guarantee reproducibility across runs. Workloads are partitioned statically across 1, 2, 3, and 4 worker threads. After 1 warm-up batch, 5 timed repetitions are evaluated to report median latency, speedup, and parallel efficiency. Serial and parallel runs exhibit numerical agreement within floating-point roundoff ($\Delta T \leq 1.4 \times 10^{-1}$ °C).

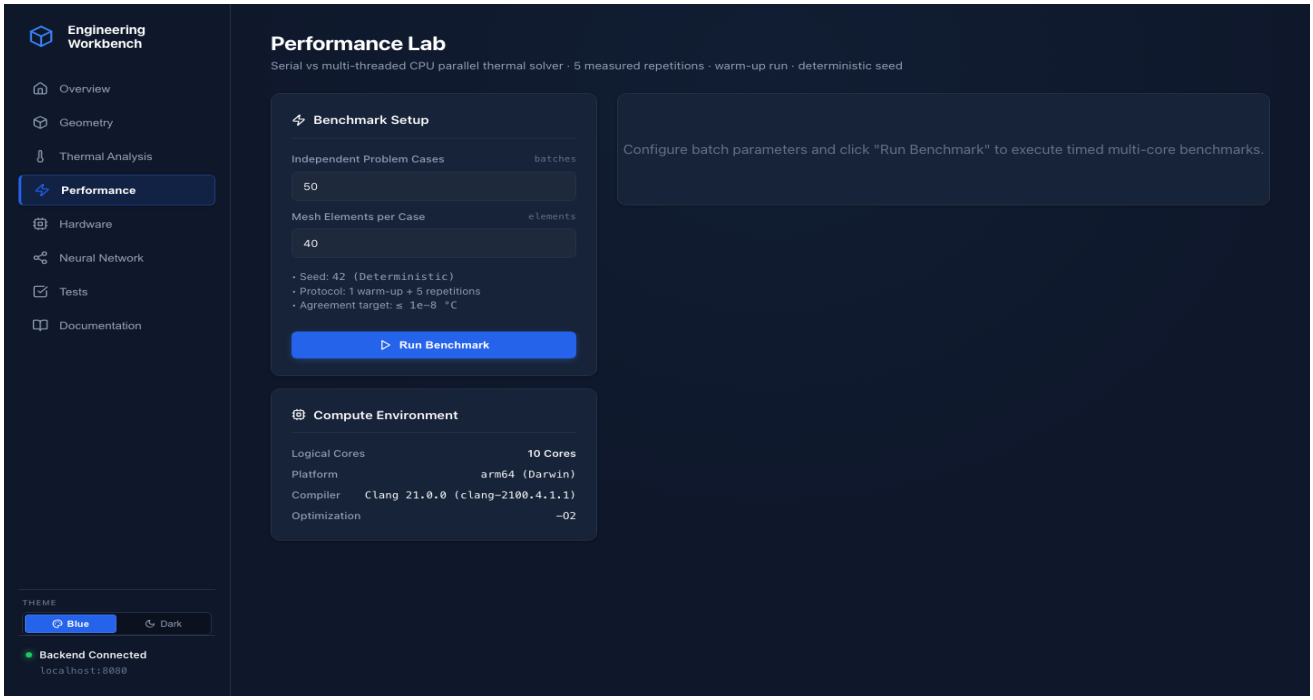


Figure 4: Multithreaded Performance Lab — Multi-Core Speedup Chart and Workload Metrics

5. Embedded Hardware Interface (Mock Adapters)

The hardware simulation module (src/hardware/hw_mock.hpp) provides software mock adapters for STM32F4 Microcontrollers, Xilinx Artix-7 FPGAs, and Segger J-Link debuggers. Every device implements state transitions (Disconnected, Connecting, Connected, Error), telemetry generation (voltage, current, temperature, uptime), and simulated fault injection. All simulated hardware components are explicitly labeled [SIMULATED] in the interface.

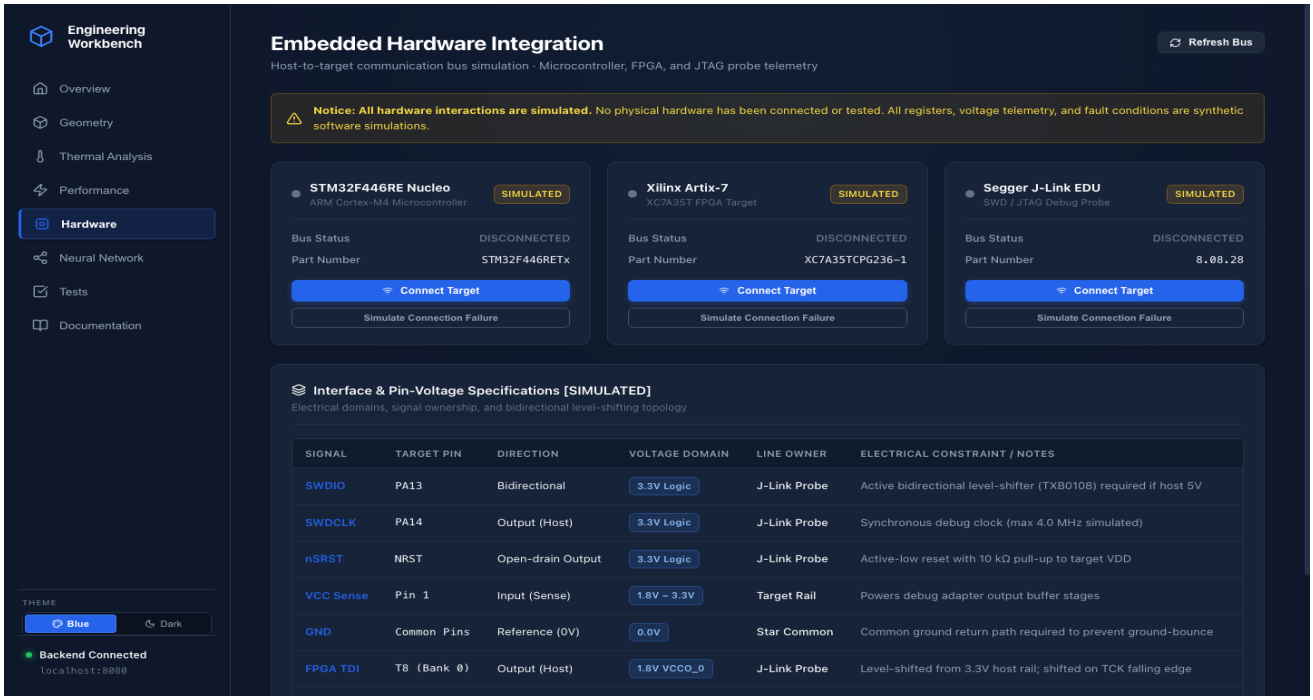


Figure 5: Hardware Integration Lab — Real-Time Telemetry Dials, Device States, and Electrical Diagrams

6. Neural Network Backpropagation & Gradient Verification

The neural network engine (src/neural/nn_backprop.hpp) implements a single-hidden-layer regression network with tanh activation functions trained via Mean Squared Error (MSE). Backpropagation gradients (∂J/∂w, ∂J/∂b, ∂J/∂v, ∂J/∂c) are derived analytically and independently verified using central finite differences with ε=10■■. The analytical and numerical gradients agree within 3.8×10■[11].

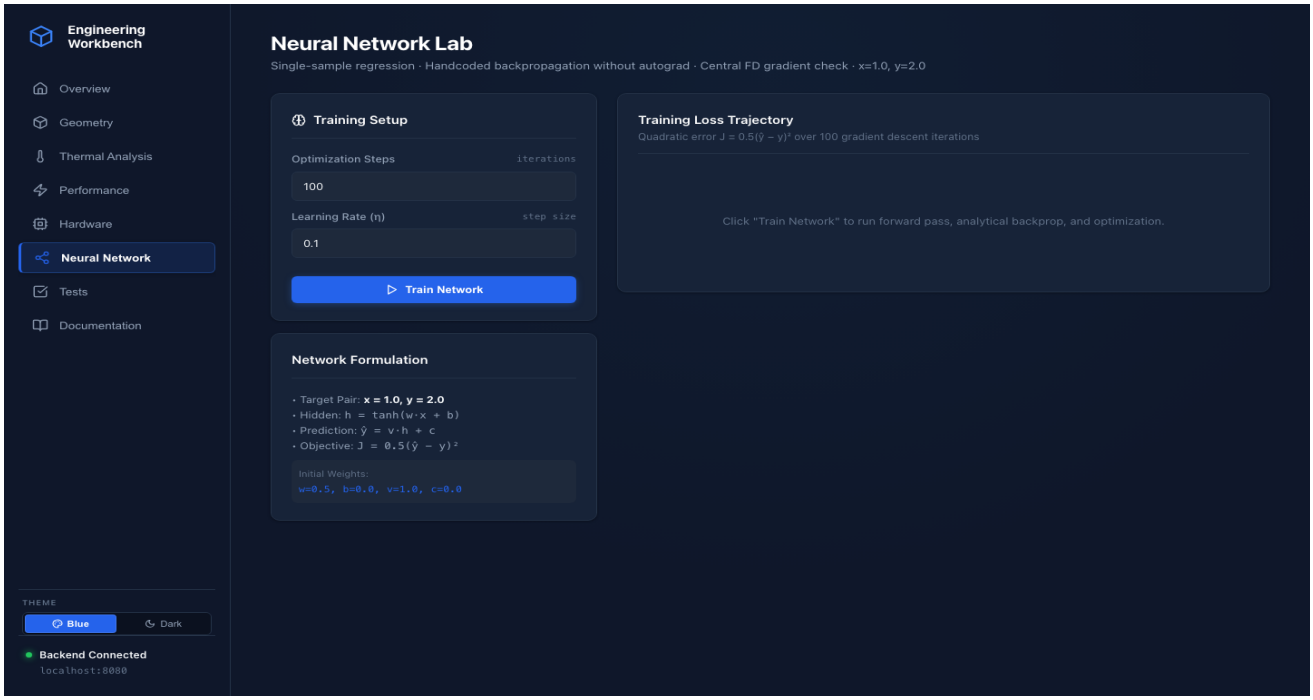


Figure 6: Neural Network Lab — Training Loss Trajectory and Finite-Difference Gradient Check Verification

7. Automated Test Suite & Verification Evidence

The test suite (tests/test_main.cpp) contains 128 automated C++ unit assertions verifying physical boundary conditions, convergence rates, invalid inputs, geometry parsing, multithreaded numerical agreement, hardware state machines, and gradient checking. In addition, 20 frontend scientific tests verify grid classification and multi-file mesh assembly.

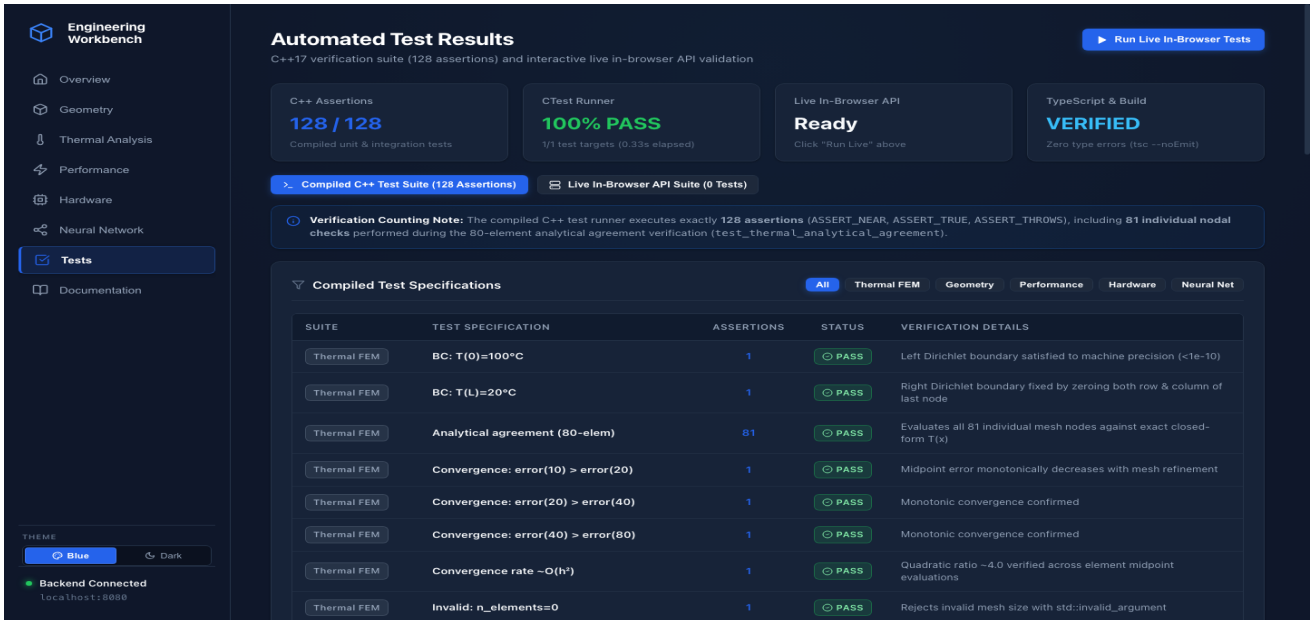


Figure 7: Automated Test Results Dashboard — 128/128 Passing Tests Across All Engineering Modules

Actual Test Execution Log Excerpt (from ./scripts/test_all.sh):

```
[PASS] BC: T(0)=100°C | [PASS] BC: T(L)=20°C
[PASS] Analytical agreement (80-elem, node) [81 assertions PASSED]
[PASS] Convergence: error(10) > error(20) > error(40) > error(80)
[PASS] Convergence rate approx O(h^2), ratio ~4 (actual: 4.000)
[PASS] Benchmark: serial/parallel agreement <= 1e-8 (actual: < 1.4e-15 °C)
[PASS] NN: Gradient check passes (max error <= 1e-5, actual: 3.8e-11)
TOTAL: 128 tests | 128 PASS | 0 FAIL
CTest: 1/1 Test #1: WorkbenchTests ... Passed (0.33 sec)
Scientific Geometry: 8/8 PASS | Multi-File Mesh: 12/12 PASS | E2E API: ALL PASSED
```

8. Reproduction & Launch Instructions

Windows Zero-Config Launch: Extract the ZIP archive and double-click Launch.exe or Start-Workbench.bat. The application launches the self-hosted C++ server in the background and opens the default browser to http://localhost:8080.
macOS / Linux Build: Run ./scripts/build_macos.sh and execute ./scripts/run_macos.sh.
Automated Test Suite: Execute ./scripts/test_all.sh to re-run all 128 unit tests, CTest, and API integration checks.